

MapReduce Example: World Temperature

(without combiners)

Mahmoud Parsian

Ph.D. in Computer Science

Table of Contents

1. Where the Full Derivation Already Lives
2. The Problem
3. The Mapper: One Record, Two Keys
4. Worked Example
5. Try It Yourself
6. Next

1. Where the Full Derivation Already Lives

This exact problem — average temperature per city — is already worked out step by step, key by key, in:

[mapreduce_examples/MapReduce_Find_Average_Temperature.md](#)

(Sections 1–9)

This deck instead walks through a **second, standalone worked example** with its own numbers — useful on its own, and as the baseline the companion deck,

[09_mapreduce_example_with_combiners.md](#),

extends with a combiner. Both should land on the same final answer.

2. The Problem

Input record: `<country>,<city>,<temperature>`

```
USA,Cupertino,73  
USA,Cupertino,73  
CANADA,Toronto,29  
CANADA,Toronto,48  
INDIA,Mumbai,68
```

Find the **average temperature per city** — and, from the same pass, the average temperature **per country** too.

3. The Mapper: One Record, Two Keys

Each input record contributes to *two* running averages at once — its city's, and its country's:

```
def map(key, value):  
    country, city, temperature = value.split(",")  
    if temperature >= 0:  
        # a mapper-side filter  
        emit(f"{country},{city}", temperature)  
        emit(country, temperature)
```

"USA,Cupertino,58" → emits both ("USA,Cupertino", 58) and ("USA", 58) — one record, two keys, computed in a single pass.

4. Worked Example

USA,Cupertino,58	USA,Cupertino,78
USA,Cupertino,67	INDIA,Mumbai,90
USA,Sunnyvale,88	INDIA,Mumbai,96
USA,Sunnyvale,77	INDIA,Agra,98
	INDIA,Agra,92

Worked Example: Mapper Output

("USA,Cupertino", 58)	("USA", 58)
("USA,Cupertino", 67)	("USA", 67)
("USA,Sunnyvale", 88)	("USA", 88)
("USA,Sunnyvale", 77)	("USA", 77)
("USA,Cupertino", 78)	("USA", 78)
("INDIA,Mumbai", 90)	("INDIA", 90)
("INDIA,Mumbai", 96)	("INDIA", 96)
("INDIA,Agra", 98)	("INDIA", 98)
("INDIA,Agra", 92)	("INDIA", 92)

Worked Example: Sort & Shuffle Output

6 unique keys, (key, [values]) :

```
("USA,Cupertino", [58, 67, 78])  
("USA,Sunnyvale", [88, 77])  
("USA", [58, 67, 78, 88, 77])  
("INDIA,Mumbai", [90, 96])  
("INDIA,Agra", [98, 92])  
("INDIA", [90, 96, 98, 92])
```


Worked Example: Reducer Output

```
def reduce(key, values):  
    emit(key, sum(values) / len(values))
```

```
("USA,Cupertino", 67.67)  
("USA,Sunnyvale", 82.5)  
("USA", 73.6)  
("INDIA,Mumbai", 93)  
("INDIA,Agra", 95)  
("INDIA", 94)
```

5. Try It Yourself

A third partition arrives with more records:

```
CANADA,Toronto,29  
CANADA,Toronto,48  
CANADA,Toronto,61
```

What does `reduce()` emit for `"CANADA,Toronto"` and for `"CANADA"` ?
(Same mapper/reducer as above — work it through, then check below.)

```
("CANADA,Toronto", (29+48+61)/3) -> 46.0  
("CANADA",         (29+48+61)/3) -> 46.0  
# only one city so far, so equal
```

Try It Yourself: MEDIAN, Not Just AVERAGE

The Sort & Shuffle step already grouped every raw value by key — nothing was pre-summarized. So the reducer can compute *any* aggregate from that list, not just **AVERAGE** .

From the **Worked Example: Sort & Shuffle Output** slide, compute the **median** for each of the 6 keys. Then compare against the **AVERAGE** reducer's output shown earlier.

Try It Yourself: MEDIAN — Answer

```
("USA,Cupertino", median=67)      # avg was 67.67 – close
("USA,Sunnyvale", median=82.5)    # avg was 82.5 – same (n=2)
("USA", median=77)                # avg was 73.6 – genuinely different!
("INDIA,Mumbai", median=93)       # avg was 93 – same (n=2)
("INDIA,Agra", median=95)         # avg was 95 – same (n=2)
("INDIA", median=94)             # avg was 94 – same (n=4, symmetric)
```

`("USA", ...)` is the interesting one: 5 raw values, genuinely different median vs. average. This works *only* because there's no combiner here — every raw value survives to the reducer. Add a combiner and this same `median()` reducer silently breaks — see [07_combiners_in_mapreduce.md](#)'s "Hard Case 1: MEDIAN".

Try It Yourself: The Filter in Action

The third partition (previous exercise) also includes a bad sensor reading:

```
CANADA,Toronto,29  
CANADA,Toronto,48  
CANADA,Toronto,61  
CANADA,Toronto,-10      # faulty reading
```

Trace `map()`'s `if temperature >= 0` check on the `-10` record — does it reach the reducer? What does `reduce()` now emit for `"CANADA,Toronto"` ?

Try It Yourself: The Filter in Action — Answer

```
map(): -10 fails the filter, never emitted at all  
reduce("CANADA,Toronto", [29,48,61]) -> 46.0    # unchanged
```

The bad reading is dropped at the **mapper**, before shuffle — see [05_filters_in_mapreduce.md](#) for the general mapper-vs-reducer filter placement rule.

6. Next

- Filtering by temperature or by average — same rules as always:
[05_filters_in_mapreduce.md](#)
- The same base dataset (extended with a third partition, to show combiners working across multiple partitions), **redone with a combiner** — and why a naive combiner would silently break an average: [09_mapreduce_example_with_combiners.md](#)